CH 2 — Classes

Luke Lu • 2026-10-01

Intro to Class and Objects

> 📘 **Definition — Classes and Objects**
>
> Imperative Programming
>
> - Procedural Programming: data + standalone functions
> - Object-Oriented Programming: put the data and the functions that operate that data within unit or construct
>   - The construct is called **class**
>   - An instance/value of a class is called an **object**

Example:

A simple class called Student(Program "06-classes/version1.cc/student.cc"):

```
struct Student{
  int assignment, mt, final;
  float grade();
}

float Student::grade(){ //In the scope of student Class, the function grade
   return 0.4 * assignment + 0.2 * mt + 0.4 * final;
}
```

Note: in C++, `struct` can have functions within, and C++ `structs` are classes

Program "06-classes/version2.cc" is the more formal and preferred way of usage.

```
Student billy{90, 55, 45}; //C-style  initialization
//billy is a student object

float res1 = billy.grade(); //grade() has access to billy's marks

Student lily{95, 80, 85};

cout << lily.grade(); //grade() has access to lily's mark
```

> 📘 **Definition — Methods**
>
> A "function" part of a class is called "**methods**", which is commonly used
>
> A **method** can only be called by using an **object**
>
> Within the **method**, the method has access to the fields of that object.
>
> All **methods** have a hidden parameter named **`this`**, which is a pointer to the object use to call the method

Example:

```
billy.grade(); //"this" is equiv to "&billy", or interpret that their address is same
```

For the function, `grade` is actually

```
 float grade(){
  return this->assignment * 0.5 + this->mt * 0.2 + this->final 0.4;
};
```

We assume `this->` if not present and identifier undeclared

Object Initialization

In C++, we can write special methods to initialize objects

Previously,

1. C-style Initialization if there is no explicit constructor
   - Memory allocation
   - Field initialization in declaration order with parameter values in declaration order
     - In case of under-initialization, system does default initialization

       non-class/primitive data $\Longrightarrow$ assigned to `0`
2. C++ style Initialization if there is at least one explicit constructor
   - Memory allocation
   - Field initialization in declaration order
     - Non-class are not initializated
     - Classes are being called with **default constructors**
   - **Constructor** calls
     - Cannot default initialize in C-style
     - Logging
     - Constructor/function overloading
     - Sanity checks, restriction on input

> 📘 **Definition — Constructors Rules**
>
> - Same name as the class name
> - No return type
> - Default constructor does not have brackets
>
> ```
> constructor(); //ILLEGAL
> constructor; //LEGAL
> ```

```
struct Student{
  //...
  Student(int assign, int mt, int final){
    this->assign = assign;
    this->mt = mt;
    this->final = final;
  }
}
```

Member-Initialization List:

- field insitialization in declaration order of C++ style for fields named in the list
- use member-initilization list for all initializations
- use constructor for sanity checks, logging, etc.
- MIL processed in field declaration order. Not in MIL order.

```
struct id{
  string name;

  id;
  id(string name){
    this->name = name;
  }
}

struct posn{
  int x, y;
  id uid;

  posn(int x, int y, string name): y{y*2}, x{x+1}, uid{name+ "?"} {}
}
```